

Ordenamiento de burbuja en C++

CONCEPTOS PREVIOS.

El **método de ordenamiento de burbuja** (o *Bubble Sort* en inglés) es uno de los algoritmos más sencillos y fundamentales en ciencias de la computación para organizar elementos (como números o palabras) dentro de una lista o arreglo en un orden específico, ya sea de menor a mayor (ascendente) o de mayor a menor (descendente).

¿En qué consiste?

Su nombre proviene de la forma en que los elementos "flotan" hacia su posición correcta: los valores más grandes o "pesados" van descendiendo hacia el final de la lista en cada pasada, mientras que los valores más pequeños o "livianos" van subiendo progresivamente hacia el principio, como si fueran **burbujas en el agua**.

1. ¿Qué es un arreglo y cómo se implementa en C++?

Un **arreglo** (o *array*) es una estructura de datos lineal que permite almacenar múltiples elementos del **mismo tipo** en posiciones de memoria continuas. Se accede a cada elemento mediante un **índice** numérico que comienza siempre en 0.

B) Arreglos dinámicos (std::vector)

Es la forma recomendada en C++ moderno. Su tamaño puede cambiar dinámicamente durante la ejecución.

2. Cómo implementar un algoritmo de comparación en C++

Para comparar dos valores en C++ se utilizan los **operadores relacionales**:

Operador	Significado	Ejemplo
>	Mayor que	<code>a > b</code>
<	Menor que	<code>a < b</code>
>=	Mayor o igual que	<code>a >= b</code>
<=	Menor o igual que	<code>a <= b</code>
==	Igual a	<code>a == b</code>
!=	Diferente de	<code>a != b</code>

3. Cómo intercambiar valores en un arreglo en C++

Para intercambiar dos elementos dentro de un arreglo existen dos métodos estándar:

Método A: Usando una variable temporal

Consiste en guardar uno de los valores en una variable auxiliar para no sobrescribirlo antes de completar el intercambio.

Método B: Usando `std::swap` (Librería estándar de C++)

La librería <utility> (incluida automáticamente en <iostream>) proporciona la función `std::swap`, que realiza el intercambio de forma limpia y óptima.

PREGUNTAS

1. ¿En qué consiste el algoritmo de ordenamiento de burbuja?

Es un algoritmo de ordenamiento comparativo directo. Consiste en recorrer repetidamente una lista o arreglo, comparar parejas de elementos contiguos e intercambiarlos si están en la posición incorrecta. Este proceso se repite en varias "pasadas" hasta que no se requiera ningún intercambio más, garantizando que todos los elementos queden ordenados.

2. ¿Por qué se comparan elementos consecutivos?

Se comparan elementos adyacentes porque el algoritmo utiliza un enfoque de **ordenamiento local**. Al verificar y corregir el orden entre vecinos inmediatos (`arreglo[j]` y `arreglo[j+1]`), se asegura que los valores más grandes (o más pequeños) vayan "desplazándose" gradualmente a lo largo de la estructura de datos, paso a paso, hasta su posición final.

3. ¿Qué sucede cuando dos elementos están en el orden incorrecto?

Se realiza un **intercambio** (*swap*). El valor del primer elemento pasa a la posición del segundo y viceversa. Para esto, en C++ se puede utilizar la función `std::swap` o una variable auxiliar temporal para

evitar perder el valor original de una de las celdas durante el reemplazo.

4. ¿Por qué se necesitan dos ciclos for?

- **Ciclo externo (i):** Controla el **número de pasadas** que se realizan sobre el arreglo. Cada pasada asegura la colocación definitiva de al menos un elemento en su posición final.
- **Ciclo interno (j):** Realiza el **recorrido elemento por elemento** en la pasada actual, haciendo las comparaciones e intercambios adyacentes.

5. ¿Qué función cumple la variable i?

La variable *i* actúa como el **contador de pasadas**. Además de llevar la cuenta de cuántas veces se ha recorrido el arreglo, en cada iteración indica cuántos elementos ya han alcanzado su posición final al final de la lista, sirviendo de límite para el ciclo interno.

6. ¿Qué función cumple la variable j?

La variable *j* actúa como el **índice de posición actual** dentro del ciclo interno. Indica la posición del primer elemento del par que se está comparando en ese instante (`arreglo[j]` frente a `arreglo[j+1]`).

7. ¿Por qué en cada pasada se reduce el número de comparaciones?

Porque al término de cada pasada *i*, se garantiza que el elemento más grande de la sublista evaluada ha "flotado" hasta su posición correcta al final del arreglo. Por lo tanto, en las siguientes pasadas ya no es necesario volver a evaluar los últimos *i* elementos, fijando la condición del ciclo interno como $j < n - i - 1$.

8. ¿Qué elemento queda colocado al final después de una pasada completa?

En un ordenamiento ascendente, al terminar la primera pasada completa, el **elemento con el valor máximo** de todo el arreglo queda fijado de manera garantizada en la última posición. En las pasadas posteriores, los siguientes valores más grandes se van asentando secuencialmente antes de él.

9. ¿Qué ocurre si el arreglo ya está ordenado?

Si el algoritmo no incluye ninguna optimización, continuará realizando todas las comparaciones y pasadas posibles ($O(n^2)$) aunque no modifique ningún valor. Sin embargo, si incluye la bandera de optimización, detectará en la primera pasada que no hubo intercambios y finalizará la ejecución inmediatamente.

10. ¿Para qué sirve la optimización mediante la variable intercambio?

Sirve para **interrumpir el algoritmo de forma anticipada** si el arreglo se ordena antes de agotar todas las pasadas teóricas. Si en una pasada completa la variable `huboIntercambio` se mantiene en `false`, significa que la lista ya está perfectamente ordenada y no tiene sentido continuar ejecutando los ciclos restantes.

11. ¿Qué diferencia existe entre ordenar ascendentemente y descendentemente?

La única diferencia técnica reside en el **operador de comparación** en el ciclo interno:

- **Ascendente (de menor a mayor):** Se intercambia si `arreglo[j] > arreglo[j + 1]`.
- **Descendente (de mayor a menor):** Se intercambia si `arreglo[j] < arreglo[j + 1]`.

12. ¿Qué significa que Bubble Sort tenga una complejidad de $O(n^2)$?

Significa que el **tiempo de ejecución crece de forma cuadrática** con respecto al número de elementos (n). El número máximo de comparaciones requeridas es aproximadamente $\frac{n(n-1)}{2}$. Por ejemplo, si el tamaño del arreglo se multiplica por 10, el número de operaciones requeridas se multiplica por $10^2 = 100$.

13. ¿Qué sucede con el número de operaciones cuando aumenta considerablemente la cantidad de elementos?

El número de operaciones **se dispara drásticamente**.

Elementos (n) Operaciones aproximadas (n^2)

10	≈ 100
1,000	$\approx 1,000,000$
100,000	$\approx 10,000,000,000$

Esto provoca una degradación severa del rendimiento, haciendo que el programa tarde segundos, minutos o incluso horas en procesar conjuntos de datos medianos o grandes.

14. ¿Qué ventajas tiene Bubble Sort?

1. **Simplicidad:** Es sumamente fácil de comprender, explicar e implementar.
2. **In-place (En su lugar):** Requiere una cantidad insignificante de memoria adicional ($O(1)$) porque ordena los elementos sobre el propio arreglo.
3. **Estabilidad:** Es un algoritmo estable; no altera el orden relativo de los elementos que tienen valores idénticos.
4. **Detección rápida de orden:** Con la variable de optimización, en un arreglo previamente ordenado solo requiere $O(n)$ comparaciones.

15. ¿Qué desventajas tiene Bubble Sort?

1. **Ineficiencia drástica:** Su tiempo de ejecución $O(n^2)$ lo hace inviable para casi cualquier uso práctico en software real.
2. **Exceso de escrituras en memoria:** Realiza un número extremadamente elevado de intercambios en comparación con otros algoritmos como *Selection Sort* o *Insertion Sort*.

16. ¿Utilizarías Bubble Sort para ordenar millones de elementos? Justifica tu respuesta.

No, jamás.

Justificación: Para ordenar 1,000,000 de elementos, Bubble Sort requiere realizar aproximadamente $1,000,000^2 = 1,000,000,000,000$ (1 billón) de comparaciones. En una computadora estándar esto tomaría **varias horas o incluso días**.

En su lugar, se utilizan algoritmos con complejidad $O(n \log n)$ como **QuickSort**, **MergeSort** o **HeapSort** (o las funciones optimizadas del lenguaje como `std::sort` en C++), los cuales procesan un millón de elementos en cuestión de milisegundos (requiriendo únicamente unas 20,000,000 de operaciones).